

MathScript

ভাইভা / ফাইনাল পরীক্ষার জন্য সহজ বাংলা Short Note

মূল নোটের ভিত্তিতে: Operator Precedence, Function Evaluation, AST, TAC এবং Compiler Concepts

এক লাইনে পুরো প্রজেক্ট: **Lexer** টোকেন বানায় -> **Parser structure/AST** বানায় -> **Evaluator** AST দেখে হিসাব করে -> **TAC/Code Generator** একই AST থেকে intermediate/ C code দেখাতে পারে।

এই নোটে কী শিখব

- MathScript কীভাবে expression বুঝে ও evaluate করে
- Operator precedence + associativity কীভাবে কাজ করে
- `sqrt()`, `pow()`, `abs()`, `sin()`, `cos()`, `tan()`, `log()` কীভাবে কাজ করে
- Lexer, Parser, AST, Semantic Analysis, Symbol Table, TAC, Code Generation সহজ ভাষায়
- ভাইভাতে সরাসরি বলার মতো ছোট উত্তর

১. MathScript-এর পুরো Flow

ধাপ	কাজ	এই প্রজেক্টে
১. Lexical Analysis	characters -> tokens	Flex, src/lexer.l
২. Parsing / Syntax Analysis	tokens -> grammar structure	Bison, src/parser.y
৩. AST	expression-কে tree আকারে রাখা	src/ast.h / ast.c
৪. Evaluation	AST traverse করে result বের করা	src/eval.c
৫. TAC	AST থেকে 3-address code print	src/ir.c
৬. C Code Generation	AST থেকে আলাদা C program তৈরি	src/codegen.c

মুখস্থ বলার উত্তর: “Lexer শুধু pieces চিনে, Parser structure ঠিক করে, AST সেই structure রাখে, আর Evaluator সেই tree follow করে result বের করে।”

২. Precedence ও Associativity

Precedence = কোন operator আগে bind করবে। Associativity = একই precedence-এর repeated operator কোন দিক থেকে group হবে।

Priority	Operator	Rule
কম	< > >= <= == !=	Left
	+ -	Left
	* /	Left
	Unary -	UMINUS
বেশি	^	Right

parser.y-তে order নিচ থেকে নয়, declaration-এর top থেকে bottom-এ গেলে later declaration বেশি precedence পায়। তাই ^ সবচেয়ে শক্তিশালী।

```
%left '<' '>' GE LE EQ NE | %left '+' '-' | %left '*' '/'
%right UMINUS | %right '^'
```

সহজ উদাহরণ

Expression	কীভাবে group হয়	Result
$2 + 3 * 4$	$2 + (3 * 4)$	14
$(2 + 3) * 4$	$(2 + 3) * 4$	20
$10 - 3 - 2$	$(10 - 3) - 2$	5
2^3^2	$2^{(3^2)}$	512
-2^2	$-(2^2)$	-4
$(-2)^2$	$(-2)^2$	4

গুরুত্বপূর্ণ: Lexer বলে না “* আগে হবে”। Lexer শুধু token দেয়। Parser precedence rules ব্যবহার করে correct tree বানায়।

৩. Shift / Reduce খুব সহজে

যখন parser-এর সামনে conflict আসে, তখন সাধারণত দুই কাজ হতে পারে: Shift = নতুন token নিয়ে এগোনো; Reduce = এখনকার অংশকে expression ধরে নেওয়া।

অবস্থা	কী হবে
token-এর precedence বেশি	Shift
rule-এর precedence বেশি	Reduce
equal + %left	Reduce
equal + %right	Shift

উদাহরণ: $2 + 3 * 4$ -এ $2+3$ দেখে “reduce” করা যেত, কিন্তু সামনে * এর precedence বেশি। তাই parser Shift করে এবং $3*4$ আগে group করে।

৪. sqrt() / Function Evaluation

sqrt(25) গেলে পুরো journey:

Step	কী হয়
১. Lexer	IDENTIFIER("sqrt"), (, NUMBER(25),)
২. Parser	IDENTIFIER + (+ arglist +) দেখে function-call structure বানায়
৩. AST	CALL node: name=sqrt, argc=1, argument=25
৪. Evaluator	call_function() নামে helper-এ পাঠায়
৫. Arity check	sqrt-এর জন্য argc অবশ্যই 1
৬. Domain check	argument < 0 হলে error
৭. Math library	C-এর sqrt() call হয়
৮. Result	sqrt(25) = 5

```
sqrt(25)
IDENTIFIER("sqrt") ( NUMBER(25) )
```

Viva line: “আমি language-level parsing, AST, argument checking, domain checking ও dispatch করেছি; numerical square-root calculation C math library করে।”

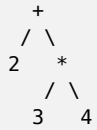
৫. Supported Built-in Functions

Function	Arg	কাজ	Example
sqrt(a)	1	non-negative square root	sqrt(25)=5
abs(a)	1	absolute value	abs(-5)=5
sin(a)	1	sine, radians	sin(pi/2)≈1
cos(a)	1	cosine, radians	cos(0)=1
tan(a)	1	tangent, radians	tan(0)=0
log(a)	1	natural log; a>0	log(e)≈1
pow(a,b)	2	a^b	pow(2,3)=8

pow(2, 3^2) = 2^9 = 512। pow-এর দুই argument আগে evaluate হয়।

৬. AST ও TAC

AST (Abstract Syntax Tree) = expression-এর meaningful tree। যেমন 2 + 3 * 4-এর root হলো +, তার right child হলো *।



এই AST থেকে evaluator result 14 বের করে। একই AST থেকে TAC print করা যায়:

```

t1 = 3 * 4
t2 = 2 + t1
RESULT = t2
  
```

নোট: বর্তমান MathScript evaluator printed TAC পড়ে result বের করে না; evaluator এবং C generator AST-ই traverse করে।

৭. Error Types

Error	উদাহরণ	অর্থ
Lexical	10 @ 5	@ token চিনতে পারেনি lexer
Syntax	10 + * 5	tokens valid structure বানাতে পারেনি
Semantic	foo(5)	syntax ঠিক, কিন্তু function supported নয়
Semantic	sqrt(1,2)	function আছে, argument count ভুল
Mathematical	sqrt(-1)	real square root-এর domain ভুল
Mathematical	10/0	division by zero

Undefined variable-ও semantic error: variable symbol table-এ পাওয়া না গেলে error।

৮. Compiler Concepts - ভাইভায় খুব সহজে বলবেন

Concept	সহজ ভাষায়	MathScript-এ
Compiler	source code বুঝে executable/target form-এ নেওয়ার system	পুরো pipeline

Lexical Analysis	characters দেখে token বানানো	Flex lexer.l
Token	code-এর ছোট recognized piece	NUMBER, IDENTIFIER, +, ^ ইত্যাদি
Lexer	tokenizer	Flex
Syntax Analysis	tokens-এর grammar/structure check	Bison parser
Parser	grammar দিয়ে structure বানায়	parser.y
Syntax Table / Parse Table	parser-এর state অনুযায়ী action/goto সিদ্ধান্তের টেবিল	Bison build-time parser tables
AST	meaningful structure-এর tree	ast.c
Semantic Analysis	meaning ঠিক আছে কিনা check	function name, argc, variable, domain checks
Symbol Table	variable/constant name থেকে value খোঁজার table	syntab_lookup
IR	target code-এর আগের intermediate form	TAC
TAC	এক লাইনে এক basic operation	t1=3*4
Code Generation	target language code তৈরি	C codegen
Runtime/Evaluation	AST থেকে actual calculation	eval.c

৯. Semantic Analysis - কী বুঝাবেন?

Syntax ঠিক থাকলেই program valid হয় না। Meaning-ও valid হতে হবে। MathScript-এ evaluation-এর আগে/সময় যেমন check হয়:

- `foo(5)` -> unknown function
- `sqrt(1,2)` -> wrong number of arguments (arity)
- `sqrt(-1)` -> invalid real domain
- Undefined variable -> symbol table-এ নেই
- `10/0` -> mathematical error

ভাইভা উত্তর: "Semantic analysis checks whether a syntactically correct expression is meaningful according to the language rules."

১০. 15টি Common Viva Q&A

- 1) **Precedence** কোথায়? src/parser.y-তে %left, %right, %prec declarations-এ।
- 2) **Lexer** কি **multiplication** আগে করে? না। Lexer token দেয়; Parser grouping ঠিক করে।
- 3) **Associativity** কী? একই precedence-এর operator কীভাবে group হবে।
- 4) $2^3^2 = 512$ কেন? ^ right-associative, তাই $2^{(3^2)}$ ।
- 5) $-2^2 = -4$ কেন? power unary minus-এর চেয়ে বেশি precedence পায়।
- 6) **Bison** কি **calculation** করে? না, এই project-এ Bison actions AST node বানায়; eval.c হিসাব করে।
- 7) **sqrt** কি **lexer keyword**? না, IDENTIFIER হিসেবে আসে; evaluator name দেখে চিনে।
- 8) **sqrt expression** নিতে পারে কেন? argument নিজেই AST; আগে recursively evaluate হয়।
- 9) **argc** কেন? function-এর expected argument count check করার জন্য।
- 10) **ok flag** কেন? 0 একটি valid result হতে পারে, তাই error আলাদা করে signal করতে।
- 11) **abs**-এ **fabs** কেন? MathScript floating-point value ব্যবহার করে।

পরীক্ষা/ভাইভা রিভিশন নোট

12) sin/cos degree? না, radians।

13) log কোন **base?** natural log, base e।

14) TAC কোথা থেকে? AST traverse করে print হয়।

15) একটি limitation? implicit multiplication + power grouping-এ limitation আছে; coefficient-এর জন্য $3*x^2$ লেখা safer।

১১. 5-Minute Demo Sequence

```
/precedence
/calc
2 + 3 * 4
(2 + 3) * 4
-2^2
2^3^2
sqrt(9 + 16)
sqrt(25) + abs(-5)
pow(2,3)
sin(pi/2)
log(e)
sqrt(-1)
sqrt(1,2)
foo(5)
gen sqrt(25) + abs(-5)
```

Demo-তে precedence দেখাতে TAC দেখান। তারপর parser.y-এর declarations এবং eval.c-এর argc/domain/function dispatch অংশ দেখান।

১২. শেষ মুহূর্তে মুখস্থ করার 6 লাইন

- Lexer -> token
- Parser -> grammar + AST
- Precedence -> কোন operator বেশি bind করবে
- Associativity -> same-level operator কোন দিকে group হবে
- Semantic Analysis -> expression-এর meaning/rules valid কি না
- Evaluator -> AST recursively evaluate করে result

সবচেয়ে শক্তিশালী এক বাক্য: “Lexer চিনে, Parser গুছায়, AST রাখে, Semantic checks validity, Evaluator হিসাব করে, TAC/Codegen একই AST থেকে অন্য representation দেখায়।”

Source note: এই PDF-এর মূল বিষয়গুলো 12 September 2026-এর MathScript study guide এবং সেখানে দেখানো source-file mapping-এর ভিত্তিতে সংক্ষেপ করা হয়েছে।